

Session 16: Pandas Series - Data Science Mentorship Program

1. Introduction to Pandas

Pandas is a fast, powerful, flexible, and easy-to-use open-source data analysis and manipulation tool built on top of the Python programming language.

Why Pandas?

- In Machine Learning, before building a model, you must understand what is happening in the data.
- Data Analysis is the process of understanding data, and Pandas is the primary tool for this.
- It allows for any kind of query, analysis, or manipulation on real-world datasets.

2. Pandas Objects: Series and DataFrames

Pandas has two main types of objects:

1. **Series:** A 1D (one-dimensional) array-like object (similar to a single column in a table).
2. **DataFrame:** A 2D tabular structure (like an entire Excel sheet or SQL table).

What is a Series?

- A Series is like a column in a table.
- It consists of two parts: **Index** and **Values**.
- It can store any data type (integers, strings, floating-point numbers, Python objects, etc.).

3. Importing Libraries

To work with Pandas, we must import it. It is also recommended to import NumPy as Pandas is built on top of it.

```
import numpy as np
import pandas as pd
```

Line-by-line Explanation:

1. `import numpy as np`: Imports the NumPy library and gives it the nickname `np`.
2. `import pandas as pd`: Imports the Pandas library and gives it the nickname `pd`.

4. Creating a Series

There are multiple ways to create a Series.

A. Creating Series from a List

```
# List of countries
country = ['India', 'Pakistan', 'USA', 'Nepal', 'Srilanka']

# Creating the Series
pd.Series(country)
```

Line-by-line Explanation:

1. `country = [...]`: Creates a standard Python list of strings.
2. `pd.Series(country)`: Calls the Series class constructor and passes the list as input to create a Series object.

Key Observation:

- When you run this, you see the values on the right and numbers (0, 1, 2...) on the left.
- The left side is the **Index** (automatically generated).
- The right side is the **Value**.
- The dtype is shown as object (Pandas calls strings "objects").

B. Creating an Integer Series

```
runs = [13, 24, 56, 78, 100]
runs_ser = pd.Series(runs)
```

Explanation: This creates a Series of integers. The dtype will be int64.

5. Custom Index and Naming

You can replace the default 0, 1, 2... index with your own labels.

```
marks = [67, 57, 89, 100]
subjects = ['maths', 'english', 'science', 'hindi']

pd.Series(marks, index=subjects, name="Nitish Ke Marks")
```

Line-by-line Explanation:

1. `marks = [...]`: List of integers representing scores.
2. `subjects = [...]`: List of strings representing the custom index.
3. `pd.Series(marks, index=subjects, name="...")`:
 - `marks`: The data values.
 - `index=subjects`: Replaces default numbers with subject names.

- `name="..."`: Assigns a name to the Series object itself.

6. Creating Series from a Dictionary

This is a very fast way because dictionary keys automatically become the index.

```
marks = {  
    'maths': 67,  
    'english': 57,  
    'science': 89,  
    'hindi': 100  
}
```

```
marks_series = pd.Series(marks, name='nitish ke marks')
```

Explanation: The dictionary keys (maths, english, etc.) become the Series index, and values become the Series data.

7. Series Attributes

Attributes give you information about the Series object without performing an operation.

1. **size:** Returns the number of items.
 - `marks_series.size` -> returns 4.
2. **dtype:** Returns the data type.
 - `marks_series.dtype` -> returns int64.
3. **name:** Returns the name of the series.
4. **is_unique:** Returns True if every value is unique, otherwise False.
5. **index:** Returns the index object (RangeIndex or Index labels).
6. **values:** Returns the values as a NumPy array.

8. Series using Real-world Datasets (read_csv)

To work with actual data, we use the `read_csv` function.

Example 1: Single Column (Subscribers Data)

```
subs = pd.read_csv('/path/to/subs.csv', squeeze=True)
```

Line-by-line Explanation:

1. `pd.read_csv(...)`: Reads a CSV file.
2. `'/path/to/subs.csv'`: The location of the file.
3. `squeeze=True`: Tells Pandas to convert a single-column DataFrame into a Series object.

Example 2: Multi-column to Series (Kohli Runs)

```
vk = pd.read_csv('kohli_ipl.csv', index_col='match_no', squeeze=True)
```

Line-by-line Explanation:

1. `index_col='match_no'`: Specifies that the 'match_no' column should be treated as the index.
2. `squeeze=True`: Since only one value column (runs) remains, it converts it to a Series.

9. Series Methods

Methods are functions performed on the Series.

A. Previewing Data

- `vk.head()`: Shows the first 5 rows.
- `vk.head(3)`: Shows the first 3 rows.
- `vk.tail()`: Shows the last 5 rows.
- `vk.sample(5)`: Shows 5 random rows (useful for checking bias).

B. `value_counts()`

Tells the frequency of each item in the series.

```
movies = pd.read_csv('bollywood.csv', index_col='movie', squeeze=True)
movies.value_counts()
```

Explanation: This will count how many movies each actor has done (e.g., Akshay Kumar: 48, Amitabh Bachchan: 45).

C. Sorting

- **`sort_values()`**: Sorts based on the data values.
 - `vk.sort_values(ascending=False).head(1)`: Finds Kohli's highest score.
- **`sort_index()`**: Sorts based on the index labels (e.g., alphabetizing movie names).
 - `movies.sort_index(inplace=True)`: Permanently sorts the movie names alphabetically.

Important Note: `inplace=True`

- Most Pandas operations are temporary. They return a copy.
- To make the change permanent in the original variable, we use `inplace=True`.

10. Mathematical/Statistical Methods

- `vk.count()`: Total number of non-missing values.
- `vk.sum()`: Total runs scored.

- `vk.mean()`: Average runs per match.
- `vk.median()`: Middle value of the score.
- `movies.mode()`: Most frequent value (Actor who did most movies).
- `vk.std()`: Standard Deviation.
- `vk.min()` / `vk.max()`: Lowest and Highest values.
- `vk.describe()`: Provides a summary of all the above statistics in one table.

11. Indexing and Slicing

Indexing

- `vk[0]` or `vk.iloc[0]`: Get value by position.
- `movies['2 States']`: Get value by label.
- **Negative Indexing**: Only works reliably if the index labels are strings. If index labels are integers, -1 will cause an error because Pandas looks for an actual label named -1.

Slicing

- `vk[5:16]`: Get matches 5 to 15.
- `vk[-5:]`: Get the last 5 matches.
- **Fancy Indexing**: `vk[[1, 3, 4, 5]]` (Passing a list of indices to get specific items).

12. Editing a Series

```
marks_series['english'] = 100 # Change existing
marks_series['sst'] = 90     # Add new item
```

Key Behavior: If the index label exists, it updates the value. If it does not exist, it adds a new row to the Series.

13. Python Logic on Series

You can use standard Python functions on Series:

- `len(vk)`: Number of rows.
- `type(vk)`: Returns `<class 'pandas.core.series.Series'>`.
- `dir(vk)`: Shows all available methods.
- `sorted(vk)`: Returns a sorted **list** of values.
- **Membership Operator**: '2 States' in movies checks the **Index**. To check values, use 'Alia Bhatt' in `movies.values`.
- **Looping**: `for i in movies:` loops through the **Values** by default.

14. Arithmetic and Relational Operators (Broadcasting)

Pandas supports broadcasting (performing an operation on every element at once).

- `100 - marks_series`: Subtracts every mark from 100.
- `vk >= 50`: Returns a Series of True (if scored 50+) and False.

15. Boolean Indexing (Filtering)

This is the most powerful way to filter data.

1. Find matches where Kohli scored > 50

```
vk[vk >= 50]
```

2. Count how many 50s he scored

```
vk[vk >= 50].size
```

3. Find actors who did more than 20 movies

```
num_movies = movies.value_counts()
```

```
num_movies[num_movies > 20]
```

Logic: We create a "mask" (the Boolean series) and pass it back into the original Series to keep only the True values.

16. Plotting

Pandas makes it very easy to visualize data using `.plot()`.

- **Line Plot:** `subs.plot()` (Good for time-series/growth data).
- **Bar Chart:** `movies.value_counts().head(20).plot(kind='bar')`.
- **Pie Chart:** `movies.value_counts().head(20).plot(kind='pie')`.

Key Takeaways (Classroom Summary)

1. **Series vs List:** Series has an index (labels) and supports vectorized operations (broadcasting), unlike lists.
2. **Missing Values:** `count()` ignores them, `size` includes them.
3. **Method Chaining:** You can write `vk.sort_values().head(1).values[0]` to get a specific value in one line.
4. **Series for DataFrames:** Every column in a DataFrame is essentially a Series. Mastering Series is the foundation for DataFrames.